

```

ellipse2).move_to(LEFT * 3)
self.play(FadeIn(ellipse_group))

i = Intersection(ellipse1, ellipse2, color=GREEN,
fill_opacity=0.5)
self.play(i.animate.scale(0.25).move_to(RIGHT * 5 +
UP * 2.5))
intersection_text = Text("Intersection", font_
size=23).next_to(i, UP)
self.play(FadeIn(intersection_text))

u = Union(ellipse1, ellipse2, color=ORANGE, fill_
opacity=0.5)
union_text = Text("Union", font_size=23)
self.play(u.animate.scale(0.3).next_to(i, DOWN,
buff=union_text.height * 3))
union_text.next_to(u, UP)
self.play(FadeIn(union_text))

e = Exclusion(ellipse1, ellipse2, color=YELLOW, fill_
opacity=0.5)
exclusion_text = Text("Exclusion", font_size=23)
self.play(e.animate.scale(0.3).next_to(u, DOWN,
buff=exclusion_text.height * 3.5))
exclusion_text.next_to(e, UP)
self.play(FadeIn(exclusion_text))

d = Difference(ellipse1, ellipse2, color=PINK, fill_
opacity=0.5)
difference_text = Text("Difference", font_size=23)
self.play(d.animate.scale(0.3).next_to(u, LEFT,
buff=difference_text.height * 3.5))
difference_text.next_to(d, UP)
self.play(FadeIn(difference_text))

```

The output of Boolean operations can be seen in Figure 5.

Best practices and optimisation

For **performance optimisation**, follow this development workflow:

- Use `-ql` flag for rapid iteration.

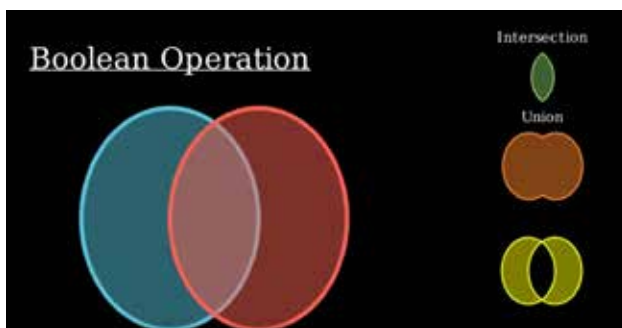


Figure 5: Boolean operations

- Cache scenes to avoid re-rendering.
- Break large projects into smaller scenes.

Organise code as follows:

```

# config.py
from manim import *

CUSTOM_BLUE = "#1E3A8A"
CUSTOM_GREEN = "#10B981"
TITLE_SCALE = 0.8

# main_animation.py
from manim import *
from config import *

```

For **collaborative development**, the version control setup should be:

- Use `-ql` flag for rapid iteration.
- Cache scenes to avoid re-rendering.
- Break large projects into smaller scenes.

Code organisation should be:

```

git init manim-project
echo "media/" >> .gitignore
echo "__pycache__/" >> .gitignore
echo "*.pyc" >> .gitignore

```

An optimised project structure is:

```

manim-project/
├── scenes/
│   ├── intro.py
│   ├── algorithms.py
│   └── conclusion.py
├── assets/
│   ├── images/
│   └── sounds/
├── config.py
└── README.md

```

Manim transforms the challenging task of creating professional animations into an accessible, code-driven workflow that aligns perfectly with open source principles. By leveraging Python's ecosystem and running natively on Linux systems, it provides animation capabilities that extend far beyond mathematical content into the programming, science, engineering, business, and creative domains. **END** 🐧

By: Mohamed Abubakkar S.

The author is a FOSS enthusiast and an upcoming associate engineer at Presidio.